

API Testing Assignment

The Movie Database (TMDB) API — v3

Base URL: `https://api.themoviedb.org/3`

Part 1: Authentication Setup [10%]

Step 1 — Create a TMDB Account

- Go to `https://www.themoviedb.org` and sign up for a free account.
- Verify your email address to activate the account.

Step 2 — Request an API Key

- Go to Settings → API from your profile menu.
- Click Create and fill in the application details.
- Note down your API Key (v3 auth) and API Read Access Token (Bearer).

Step 3 — Set Up Authentication in Postman

- Authorization Type: Bearer Token in the level of collect
- Set Environment Variables in Postman:
 - `base_url` → `https://api.themoviedb.org/3`
 - `api_key` → Your API key
 - `bearer_token` → Your Read Access Token

Step 4 — Test Authentication

Send the following GET request to verify authentication is working:

```
GET {{base_url}}/movie/550
```

Expected: Status 200 OK with details for 'Fight Club' (movie ID 550).

Part 2: API Testing Scenarios [90%]

For each scenario below: Run the request in Postman and verify all listed test points.

Scenario 1: Search Movies by Keyword [5%]	
Endpoint	GET <code>https://api.themoviedb.org/3/search/movie?query=inception&language=en-US</code>
Purpose	Verify that searching for movies by keyword returns relevant and accurate results.
Test Points	• Status code = 200 OK • Response contains results[] array • Validate that movie titles are relevant to the search keyword • Validate each result includes: id, title, release_date, overview • Confirm total_results > 0

Scenario 2: Search TV Shows by Name [5%]	
Endpoint	GET <code>https://api.themoviedb.org/3/search/tv?query=breaking+bad&language=en-US</code>
Purpose	Confirm that TV show search returns valid results for a given show name.
Test Points	• Status code = 200 OK • Response includes results[] with at least one item • Validate each result includes: id, name, first_air_date, overview • Validate results are not empty

Scenario 3: Get Movie Details by Movie ID [10%]	
Endpoint	GET <code>https://api.themoviedb.org/3/movie/{movie_id}</code>
Purpose	Retrieve detailed information for a specific movie using its TMDB ID.
Test Points	• Status code = 200 OK • Validate presence of: title, overview, release_date, runtime • Validate genres[] array is not empty • Validate production_companies[] is present • Confirm budget and revenue fields exist (may be 0 for some movies)

Scenario 4: Get Movie Credits (Cast & Crew) [10%]	
Endpoint	GET <code>https://api.themoviedb.org/3/movie/{movie_id}/credits</code>
Purpose	Retrieve the full cast and crew list for a specific movie.
Test Points	• Status code = 200 OK • Response includes cast[] and crew[] arrays • Validate each cast member includes: id, name, character, order • Validate crew members include: id, name, job, department • Confirm cast list is not empty

Scenario 5: Get Popular Movies [10%]	
Endpoint	GET <code>https://api.themoviedb.org/3/movie/popular?language=en-US&page=1</code>
Purpose	Verify that the popular movies endpoint returns a paginated list of trending movies.
Test Points	• Status code = 200 OK • Response contains results[] with 20 items per page • Validate presence of: page, total_pages, total_results • Each movie includes: id, title, vote_average, popularity • Verify results are sorted by popularity (descending)

Scenario 6: Get TV Show Details by ID [10%]	
Endpoint	GET <code>https://api.themoviedb.org/3/tv/{series_id}</code>
Purpose	Retrieve detailed information for a specific TV show using its TMDB series ID.
Test Points	• Status code = 200 OK • Validate presence of: name, overview, first_air_date, number_of_seasons • Validate genres[] is not empty • Confirm episode_run_time[] field exists • Validate networks[] and created_by[] are present

Scenario 7: Get Movie Genres List [10%]

Endpoint	GET <code>https://api.themoviedb.org/3/genre/movie/list?language=en</code>
Purpose	Test that the genre list endpoint returns all available movie genre categories.
Test Points	• Status code = 200 OK • Response includes genres[] array • Validate genres like 'Action', 'Comedy', 'Drama' are present • Each genre has: id and name fields • Confirm list is not empty

Scenario 8: Discover Movies by Genre [10%]

Endpoint	GET <code>https://api.themoviedb.org/3/discover/movie?with_genres={genre_id}&sort_by=popularity.desc</code>
Purpose	Retrieve a filtered list of movies belonging to a specific genre using the Discover endpoint.
Test Points	• Status code = 200 OK • Response contains results[] filtered by the given genre • Validate each movie includes: id, title, genre_ids[], vote_average • Confirm genre_id appears in genre_ids[] for each result • Validate pagination fields: page, total_pages, total_results

Scenario 9: Get Now Playing Movies [10%]

Endpoint	GET <code>https://api.themoviedb.org/3/movie/now_playing?language=en-US&page=1</code>
Purpose	Validate that the now playing endpoint returns movies currently in theaters.
Test Points	• Status code = 200 OK • Response contains results[] and dates{} object • dates{} includes: maximum and minimum (date range) fields • Each movie includes: id, title, release_date, vote_count • Confirm results are within the valid date range

Scenario 10: Get Person Details (Actor/Director) [10%]	
Endpoint	GET <code>https://api.themoviedb.org/3/person/{person_id}</code>
Purpose	Retrieve biographical and filmography details for a specific person (actor or director).
Test Points	• Status code = 200 OK • Validate presence of: name, biography, birthday, place_of_birth • Confirm known_for_department field exists (e.g. 'Acting') • Validate popularity score is a number • Confirm also_known_as[] array is present

Scenario 11: [POST] Add Movie to Watchlist (POST) [bonus]	
Endpoint	POST <code>https://api.themoviedb.org/3/account/{account_id}/watchlist</code>
Purpose	Add a specific movie to an authenticated user's watchlist.
Headers	Authorization: <code>Bearer {{bearer_token}}</code> Content-Type: <code>application/json</code>
Request Body	<code>{ "media_type": "movie", "media_id": 550, "watchlist": true }</code>
Test Points	• Status code = 201 Created (first time) or 200 OK (if already in watchlist) • Response contains success: true • Validate status_message confirms the item was added • Send the same request again — confirm 200 OK is returned (idempotent) • Validate media_type = 'movie' and media_id = 550 are echoed or implied in response

Submission Instructions

- 1. Create a Public GitHub repository named: TMDB_API_Assignment**
- 2. Upload:**
 - **Postman Collection (.json)**
 - **Postman Environment (.json)**
- 3. Send an email to: abdelrhman.route1@gmail.com**
 - **Subject: API Project - [Your Full Name] – G1**
 - **Body:**
 - **Full Name**
 - **Repository Link**